

설비 고장을 예측하는 AI 데이터 분석

NumPy 기초

데이터 추출과 통계 분석

posco

C O N T E N T S

목차

01 인덱싱과 슬라이싱 — 값 추출과 행·열 선택

02 배열 연산과 필터링 — 벡터 연산, 조건 필터링

03 기초 통계 — 평균·표준편차, axis 방향

공정 — CNC 가공(MCT)

MCT는 공구를 회전시켜 금속을 깎는 정밀가공 설비. 스피들 회전수·토크·온도를 기록한다.

핵심 값

회전수(스피들 속도)와 토크(깎는 힘)가 대표 지표. 정상 가공은 고회전을 유지한다.

왜 이 분석

배열(NumPy)로 회전수·토크를 한 번에 통계 내고, 회전수가 크게 떨어진 시점을 이상으로 걸러낸다.

01

인덱싱과 슬라이싱

값 추출과 행.열 선택 · 17분

1차원 인덱싱

배열을 만들었으면 그 안에서 원하는 값을 꺼낼 줄 알아야 함

01 · 값 추출의 시작

배열에서 원하는 값 하나를
꺼내는 가장 기본 방법

앞서 배열을 만들고 모양을 다뤘다면, 이제부터는 그 안에서 원하는 값
만 콕 집어 꺼내기.

02 · 인덱싱의 정의

대괄호와 번호로
특정 위치의 값 하나를 꺼내기

번호는 인덱스라고 부름. 다음 슬라이드에서 실제 코드로 확인.

1차원 인덱싱 — 번호로 값 꺼내기

번호는 0부터 시작, 음수는 뒤에서부터

CODE

```
temp = np.array([70, 72, 71, 95, 73])
print(temp[0])
# 출력: 70 (첫 번째)
print(temp[-1])
# 출력: 73 (맨 마지막)
```

1차원 슬라이싱

시작:끝으로 구간 잘라내기 (끝 번호 제외)

CODE

```
temp = np.array([70, 72, 71, 95, 73, 68])
print(temp[1:4])
# 출력: [72 71 95]
print(temp[::2])
# 출력: [70 71 73]
```

1:4는 1·2·3번 (끝 4 제외) · 콜론 둘은 간격, ::2는 두 칸씩 건너뛰기

슬라이싱의 핵심 규칙

파이썬 전체에서 통하는 두 가지 약속

RULE 01

시작은 **포함**,
끝은 **제외**

arange의 끝값 제외와 같은 약속. 파이썬 전체에서 일관됨.

RULE 02

시작·끝 **생략 가능**,
콜론 두 개로 간격 지정

생략하면 처음부터 또는 끝까지. 슬라이싱 결과 수정 시 원본도 바뀔 수 있어 주의.

2차원 인덱싱

행과 열을 함께 지정 — 앞=행, 뒤=열

CODE

```
data = np.array([[70, 2.1], [72, 2.3]])  
print(data[0, 1])  
# 출력: 2.1 (0행 1열)
```

쉽게 행·열을 한 번에 지정 · shape 순서(행, 열)와 동일

2차원 인덱싱의 순서

표 모양 배열의 위치 표현은 두 가지만 기억

01 · 순서

항상 **행이 먼저, 열이 나중**

shape에서 앞이 행 뒤가 열이었던 것과 같은 순서. 두 개념이 같은 순서를 쓰니 함께 기억.

02 · 시작 번호

번호는 0부터
왼쪽 위 칸이 0행 0열

파이썬 리스트의 대괄호 두 번 방식보다 NumPy의 쉼표 표현이 더 간결.

2차원 슬라이싱 — 행·열 선택

행 전체, 열 전체, 일부 구간 잘라내기 — **콜론이 전부를 의미**

CODE

```
data = np.array([[70, 2.1], [72, 2.3]])
print(data[0])
# 출력: 행 전체
print(data[:, 0])
# 출력: 0열 전체
```

[:, 0]은 모든 행에서 0번 열만 — 특정 센서의 전체 시점값 추출에 가장 자주 씀

행 추출과 열 추출

같은 표에서 꺼내는 방식이 다른 이유 — 행은 기본 단위, 열은 가로질러 모음

01 · 행 추출

data 대괄호에
행 번호만

그 시점의 모든 센서값. 행은 배열의 기본 단위라 번호만으로 추출 가능.

02 · 열 추출

컬론과 열 번호

그 센서의 모든 시점값. 모든 행을 가로질러 모아야 해 컬론이 필요.

인덱싱·슬라이싱 정리

목적에 따라 표현 선택 — 번호 0부터, 끝은 제외 (1·2차원 공통)

목적	표현	결과
값 하나	번호 1개 · <code>a[2]</code>	2번 값
구간	시작:끝 · <code>a[1:4]</code>	1~3번
행 전체	행 번호 · <code>data[0]</code>	0행 모두
열 전체	콜론, 열번호 · <code>data[:, 0]</code>	0열 모두

두 규칙은 1차원이든 2차원이든 동일 — 0부터 시작, 끝은 제외

자주 하는 세 가지 실수

세 가지만 조심하면 추출에서 겪는 어려움 대부분이 사라짐

MISTAKE 01

번호를 **1부터** 세기

항상 **0부터** 시작. 첫 번째 값은 1번이 아니라 0번.

MISTAKE 02

슬라이싱 끝 번호 **포함** 착각

끝은 **제외**. 1:4는 4번을 포함하지 않음.

MISTAKE 03

2차원에서 **행.열 순서** 바꿈

항상 **행이 먼저**. shape과 같은 순서로 일관 유지.

실습 1. 특정 센서·구간 추출하기

목표

회전수 배열에서 특정 시점과 일정 간격의 값 추출

단계

- 회전수 측정값 배열 준비
- 인덱싱으로 첫 시점과 마지막 시점 값 꺼내기
- 슬라이싱으로 앞 구간과 두 칸 간격 값 추출

예상 결과

특정 시점 값, 앞 구간 값, 두 칸 간격 값이 각각 출력

실습 2. 행·열 단위로 추출하기

목표

표 모양 센서 데이터에서 특정 행과 열을 단위로 추출

단계

- 여러 설비의 회전수·토크를 담은 이차원 배열 준비
- 행 인덱싱으로 특정 설비의 값 전체 꺼내기
- 열 인덱싱으로 모든 설비의 회전수 열과 토크 열 추출

예상 결과

특정 설비 행, 회전수 열, 토크 열이 각각 출력

02

배열 연산과 필터링

벡터 연산 · 조건 필터링 · 30분

배열 산술 연산

두 배열을 **같은 위치끼리** 한 번에 계산

CODE

```
a = np.array([1, 2, 3])  
b = np.array([10, 20, 30])  
print(a + b)  
# 출력: [11 22 33]
```

요소별 연산의 조건

같은 위치끼리 계산되려면 두 배열의 모양이 맞아야 함

01 · 핵심 원리

같은 위치 값끼리 계산

첫째는 첫째끼리, 둘째는 둘째끼리. 어제와 오늘 데이터를 빼서 변화량을 한 번에 구하는 방식.

02 · 조건

두 배열의 크기가 같아야

값 3개인 배열과 5개인 배열은 위치를 맞출 수 없어 오류 발생.

스칼라 연산

배열 전체에 **숫자 하나**를 한꺼번에 적용 — 섭씨→화씨 한 줄

CODE

```
celsius = np.array([20.0, 25.0, 30.0])  
print(celsius * 1.8 + 32)  
# 출력: [68. 77. 86.]
```

스칼라 연산의 특징

크기를 맞추지 않아도 — 데이터 크기 일괄 조정(스케일링)에 자주 사용

01 · 적용 범위

배열 크기와 무관하게
모든 값에 적용

5개든 백만 개든 한 줄. 요소별 연산과 달리 크기를 맞추지 않아도.

02 · 연산 순서

수학과 같은 순서
(곱하기 먼저)

더하기를 먼저 하려면 괄호로 묶기. 우선순위 규칙은 수학과 동일.

브로드캐스팅

한 줄짜리 **기준값**이 모든 행에 퍼져서 계산

CODE

```
table = np.array([[72, 2.3], [95, 6.8]])  
base = np.array([70, 2.0])  
print(table - base)  
# 출력: 각 행에서 기준값 빼기
```

각 센서가 기준에서 얼마나 벗어났는지 모든 시점에 대해 한 번에 계산

브로드캐스팅의 원리

작은 것이 큰 것 모양에 맞춰 도장처럼 **반복 적용**

01 · 숫자 하나

모든 위치로 퍼짐
(스칼라 연산)

하나의 숫자가 배열의 모든 자리에 똑같이 적용

02 · 한 줄 배열

모든 행으로 퍼짐
(기준값 빼기)

한 줄짜리 배열이 표의 모든 행에 동일하게 적용

03 · 조건

**작은 쪽 크기가
1이거나 같아야**

아무 때나 가능하지 않음. 크기가 어긋나면 오류 발생

벡터 연산 정리

반복문 없이 배열 전체를 한 번에 계산 — NumPy가 빠른 이유

종류	대상	예시
요소별	배열 + 배열	<code>a + b</code>
스칼라	배열 + 숫자	<code>a * 2</code>
브로드캐스팅	큰 + 작은	<code>table - base</code>

세 가지 공통점 — 반복문 불필요 · 데이터 분석에서 NumPy를 쓰는 가장 큰 이유

벡터 연산의 사고방식

리스트 사고에서 **배열 전체 사고**로의 전환

01 · 핵심

반복문 없이 한 줄로

데이터가 클수록 이점이 큼. 큰 데이터일수록 한 줄 처리의 효과가 두드러짐.

02 · 사고 전환

개별 값이 아니라
배열 전체를 대상으로

"모든 온도를 화씨로" → "온도 배열에 1.8 곱하고 32 더하기"로 바로 옮기기.

실습 3. 센서값 정규화하기

목표

회전수 배열을 0과 1 사이 값으로 정규화

단계

- 회전수 측정 배열 준비
- 최솟값과 최댓값을 min, max로 확인
- 정규화 공식을 브로드캐스팅으로 적용해 변환

예상 결과

가장 작은 값이 0, 가장 큰 값이 1이 되는 정규화 배열 출력

비교 연산과 불리언 배열

조건을 걸면 참거짓 배열이 만들어짐 — 같다 비교는 등호 두 개

CODE

```
vals = np.array([70, 95, 71, 88, 73])
print(vals > 85)
# 출력: [False True False True False]
```

원래 배열과 같은 크기의 참거짓 배열 · 조건에 맞는 위치만 True

불리언 배열이란

값을 골라낸 게 아니라 — 조건에 맞는 위치를 **표시**한 상태

01 · 표시 방식

조건에 맞으면 **True**
아니면 **False**

원래 자리에 참거짓이 그대로 표시. 어디가 맞는지 위치 표시 단계.

02 · 속도

비교 연산도
벡터 연산

백만 개 값도 한 줄로 비교. 거대한 센서 데이터에서 조건 맞는 위치를 순식간에 탐지.

Boolean indexing

불리언 배열로 **조건에 맞는 값만** 골라내기

CODE

```
vals = np.array([70, 95, 71, 88, 73])  
print(vals[vals > 85])  
# 출력: [95 88]
```


Boolean indexing의 힘

이상 징후 탐색의 **가장 기본 도구** — 두 단계 원리

01 · 동작 방식

대괄호 안에 조건 →
True 자리 값만 남음

원리는 두 단계 — 조건이 참거짓 배열을 만들고, 그걸로 값을 골라냄.

02 · 속도

반복문 없이 한 줄로
원하는 값 걸러내기

수백만 개 중 조건에 맞는 것만 골라내는 것도 한 줄.

np.where 조건 처리

조건에 따라 값을 **둘 중 하나로 바꾸기** — 조건·참·거짓 세 가지 인자

CODE

```
vals = np.array([70, 95, 71, 88, 73])  
print(np.where(vals > 85, 1, 0))  
# 출력: [0 1 0 1 0]
```

모든 값을 유지하면서 위험·정상 꼬리표 붙이기 — 위험은 1, 정상은 0

Boolean indexing vs np.where

골라낼 것인가, 표시할 것인가 — 둘을 가르는 기준

01 · 불리언 인덱싱

조건 맞는 값만 남김
결과 크기 **작아짐**

위험값만 보고 싶을 때 사용. 나머지는 버려짐.

02 · np.where

모든 값 유지하고 바꿈
결과 크기 **그대로**

전체에 위험·정상 꼬리표를 달고 싶을 때 사용.

다중 조건 결합

각 조건을 괄호로 묶고 & 또는 기호로 연결

CODE

```
vals = np.array([70, 95, 71, 88, 73])  
print(vals[(vals > 70) & (vals < 90)])  
# 출력: [71 88 73]
```

각 조건은 반드시 괄호로 — 빠뜨리면 오류 · 둘 중 하나만이면 세로 막대 사용

다중 조건의 규칙

초보자가 가장 자주 틀리는 부분 — 괄호 누락

01 · 필수 규칙

각 조건을 괄호로 묶기

(조건1) & (조건2)

기호가 비교보다 먼저 계산되려 해, 괄호 없이 쓰면 순서가 꼬여 오류.

02 · 두 기호

& 그리고 (둘 다)
또는 (하나만)

정상 범위는 그리고, 여러 위험 신호 중 하나는 또는을 사용.

조건 필터링 정리

비교 · 골라내기 · 바꾸기 · 묶기 — 네 가지로 거의 모든 조건 처리 가능

방법	하는 일	결과 크기
비교 연산	참거짓 판단	그대로
불리언 인덱싱	값 골라내기	줄어듦
np.where	값 바꾸기	그대로
다중 조건	조건 묶기	경우별

네 가지를 조합하면 거의 모든 조건 처리 가능

조건 필터링과 안전관제

"경험상 위험하다"를 "데이터상 몇 %가 기준을 넘었다"로 — 진짜 분석의 시작

01 · 분석의 목표

정상에서 벗어난 순간을
찾는 것

온도가 위험 수준을 넘은 시점, 진동이 비정상적으로 커진 구간 — 모두 조건 필터링.

02 · 결합의 힘

필터링 + 통계로
"전체의 몇 %가 기준 초과"

필터링과 통계가 결합될 때 진짜 분석이 시작 — 현장 지식이 데이터로 드러남.

실습 4. 이상 센서값 필터링하기

목표

조건에 맞는 이상 센서값만 불리언 인덱싱으로 선별

단계

- 회전수와 토크 배열 준비
- 비교 연산으로 회전수가 기준을 넘는 조건 생성
- 다중 조건으로 회전수 과다 또는 토크 과소 위험 시점 필터링

예상 결과

기준 초과 회전수 값과, 위험 조건을 만족하는 위치가 출력

실습 5. 조건별 개수와 비율 세기

목표

조건을 만족하는 값의 개수와 전체 대비 비율 계산

단계

- 토크 배열 준비
- 비교 조건으로 참·거짓 불리언 배열 생성
- 불리언 배열의 합으로 개수, 평균으로 비율 계산

예상 결과

조건을 만족하는 값의 개수와 비율이 출력

03

기초 통계

평균·표준편차 · axis 방향 · 30분

통계가 왜 필요한가

수천·수만 개 값을 일일이 못 보니, 몇 개의 대표 숫자로 요약하는 일

01 · 요약

수많은 값을
몇 개의 숫자로 압축

평균이 얼마인지, 값들이 얼마나 흩어졌는지 — 데이터의 특징을 한눈에.

02 · 핵심

정상을 알아야
이상을 안다

평소 평균이 70도임을 알아야 95도가 나왔을 때 "이상하다"고 판단 —
통계가 이상 탐지의 토대.

주요 통계가 알려주는 것

각 통계는 데이터의 **다른 측면**을 보여줌 — 일상에서 이미 쓰는 개념

통계	알려주는 것
평균	대표적인 중심값 — 데이터를 한 숫자로 대표
최대 · 최소	값의 범위 — 어느 폭에 퍼져 있는지
표준편차	흩어진 정도 — 값들이 평균에서 얼마나 멀어지는지

합계와 평균(mean)

전체를 더하고 개수로 나눈 **대표값**

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(s.sum())
# 출력: 381
print(s.mean())
# 출력: 76.2
```

95라는 높은 값 하나 때문에 평균이 76.2 — 대부분(70~73)보다 위

평균의 약점

평균만으로 판단하면 오해 발생 — 다른 통계와 함께 보기

01 · 약점

유난히 크거나 작은 값
(**이상치**)에 휘둘림

모든 값을 더해서 계산하니, 유난히 큰 값 하나가 합계에 그대로 반영.

02 · 사례

대부분 70~73인데
95 하나로 평균 **76.2**

실제 중심은 72 근처인데 평균이 더 위로 끌림. 평균만으론 잘못된 그림.

중앙값(median)

값을 줄 세웠을 때 **한가운데** 오는 값 — 다른 통계와 달리 np.median 형태

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(np.median(s))
# 출력: 72.0
print(s.mean())
# 출력: 76.2
```

중앙값 72는 대부분 값에 가까움 — 이상치가 있어도 한가운데는 변하지 않음

중앙값의 강점

두 통계를 함께 구해 비교하는 습관 — 데이터 건강 점검

01 · 강점

이상치에 흔들리지 않음

맨 끝 값이 95든 950이든 한가운데 위치는 그대로. 줄의 중앙은 극단값과 무관.

02 · 활용

평균과 중앙값이 크게 다르면 이상치가 있다는 신호

비슷하면 데이터가 고름 · 크게 다르면 한쪽으로 치우친 이상치 존재.

최대·최소·범위

가장 큰 값·작은 값, 그 차이로 **폭** 파악

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(s.max(), s.min())
# 출력: 95 70
print(s.max() - s.min())
# 출력: 25
```

범위가 크면 넓게 퍼짐, 작으면 안정적으로 모임 — 폭으로 안정도 판단

최대·최소의 활용과 한계

단 하나의 극단값 — 평균·표준편차와 **함께** 봐야 정확한 그림

01 · 활용

정상 범위 넘은 최댓값
= **위험했던 순간**의 신호

범위가 갑자기 커지면 값이 불안정하게 요동친 신호일 가능성.

02 · 한계

단 하나의 **극단값**이라
전체 판단에는 부족

평균이나 표준편차 같은 다른 통계와 함께 봐야 정확한 그림.

분산(var)

값들이 평균에서 **흩어진 정도**를 하나의 숫자로

CODE

```
stable = np.array([70, 71, 70, 72, 71])
unstable = np.array([60, 85, 65, 95, 70])
print(round(stable.var(), 1))
# 출력: 0.6
print(round(unstable.var(), 1))
# 출력: 174.0
```

안정적이면 분산이 작고, 불안정하면 분산이 큼

분산의 의미와 한계

분산이 갑자기 커지면 **이상 신호** — 단위 어색함은 다음에 배울 표준편차로 보완

01 · 의미

작으면 **안정**
크면 **불안정**

정상 설비는 분산이 작음. 고장 징후가 있으면 값이 요동쳐 분산이 커짐.

02 · 한계

값을 제공해 구하므로
단위가 달라짐

온도의 분산은 "도의 제곱"이라는 어색한 단위. 표준편차로 해결.

표준편차(std)

분산의 제곱근 — 원래 단위로 흩어진 정도 표현

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(round(s.var(), 2))
# 출력: 89.36
print(round(s.std(), 2))
# 출력: 9.45
```

"값들이 평균에서 약 9.45도 흩어져 있음" — 원래 단위(도)로 바로 해석

표준편차와 이상 탐지

실무에서 분산보다 표준편차를 훨씬 자주 사용 — 이상 탐지의 기초

01 · 해석성

분산을 원래 단위로
되돌린 값

해석하기 쉬워 실무에서 표준편차를 훨씬 자주 사용.

02 · 이상 탐지

평균에서 표준편차의
2~3배 벗어나면 이상치

어떤 값이 평균에서 몇 배 떨어졌는지 측정 — 이상 탐지의 기초.

평균과 표준편차, 함께 보기

평균이 같아도 표준편차가 다르면 전혀 다른 데이터

설비	평균	표준편차
설비 A	70도	1도 (안정)
설비 B	70도	15도 (불안정)

평균만 보면 똑같음 · 표준편차를 함께 보면 B의 불안정이 드러남

평균은 중심, 표준편차는 흩어짐

두 통계의 변화를 추적하는 일이 — 예지보전의 기본 아이디어

01 · 역할

평균은 중심
표준편차는 흩어짐

둘을 함께 봐야 데이터 모습이 머릿속에 그려짐.

02 · 신호

평균이 변하거나
표준편차가 커지면 상태 변화

정상은 평균이 일정·표준편차가 작음. 고장이 다가오면 둘 다 흔들림.

axis 개념 (행·열 방향)

통계를 어느 방향으로 낼지 정하기 — axis 생략 시 전체 평균

CODE

```
mat = np.array([[70, 2.1], [72, 2.3]])  
print(mat.mean(axis=0))  
# 출력: 열별(센서별) 평균  
print(mat.mean(axis=1))  
# 출력: 행별(시점별) 평균
```

axis=0과 axis=1

헛갈리면 결과 개수로 구분 — 센서별 통계가 가장 좋음 (axis=0)

01 · 가장 자주 쓰는 방향

**axis=0 — 행 따라 아래로
열별(센서별) 결과**

한 열 안의 모든 행 값을 모아 하나로 합침 — 열마다 값 하나씩 남음.

02 · 행 방향

**axis=1 — 열 따라 옆으로
행별(시점별) 결과**

2행 3열 배열에서 axis=0은 값 3개, axis=1은 값 2개.

기초 통계 함수 정리

중심 · 범위 · 흠어짐 — 세 측면을 함께 봐야 데이터 성격이 드러남

통계	함수	측면
평균	mean	중심
중앙값	np.median	중심
최대·최소	max · min	범위
표준편차	std	흠어짐

형태 — 대부분 배열 뒤에 점, 중앙값만 np.median · 표 모양은 axis로 방향 지정

통계로 데이터 읽는 흐름

네다섯 개 숫자만 구해도 데이터 성격이 거의 드러남 — 분석은 통계로 시작

STEP 1

중심 파악

평균·중앙값으로 중심 파악 (비교로 이상치 점검)

`.mean() / np.median()`



STEP 2

폭 확인

최대·최소로 데이터의 폭과 극단값 확인

`.max() / .min()`



STEP 3

흩어짐 파악

표준편차로 얼마나 흩어졌는지 파악

`.std()`

각 통계가 서로를 보완 — 함께 보는 게 좋음 · 통계는 분석의 끝이 아닌 시작

실습 6. 센서별 기초 통계 구하기

목표

표 모양 데이터에서 센서별(열별) 통계 계산

단계

- 여러 설비의 회전수·토크 이차원 배열 준비
- axis를 열 방향으로 지정해 센서별 평균 계산
- 센서별 표준편차 계산

예상 결과

회전수·토크 각각의 평균과 표준편차가 출력

실습 7. 파일 데이터로 기초 통계 구하기

목표

파일로 저장된 공정 데이터를 불러와 기초 통계 계산

단계

- np.loadtxt로 회전수 열을 파일에서 불러오기
- 불러온 배열의 평균과 표준편차 계산
- 최솟값과 최댓값으로 값의 범위 확인

예상 결과

회전수의 평균·표준편차와 최솟값·최댓값이 출력

실습 8. 필터링과 통계 결합하기

목표

조건으로 값을 골라낸 뒤 그 값들의 통계 계산

단계

- 토크 배열 준비
- 불리언 인덱싱으로 기준을 넘는 값만 추출
- 추출한 값들의 평균과 개수 계산

예상 결과

기준 초과 값들의 평균과 개수가 출력

실습 9. NumPy 기초 종합 분석

목표

데이터 불러오기, 구조 확인, 필터링, 통계를 하나의 흐름으로 수행

단계

- np.loadtxt로 회전수와 토크 두 열을 불러오기
- shape과 dtype으로 구조 확인
- 회전수가 기준 아래로 떨어진 이상 시점을 필터링해 개수와 평균 계산

예상 결과

데이터 구조, 이상 시점 개수, 이상 시점 평균 회전수가 출력

데이터 분석의 전체 흐름

이 흐름이 몸에 배면 — 어떤 데이터도 다룰 수 있음

STEP 1

불러오기

파일에서 배열로

`np.loadtxt`



STEP 2

구조 확인

shape·dtype으로 모양
점검

`.shape · .dtype`



STEP 3

추출

인덱싱·슬라이싱으로
원하는 부분만

`data[:, k]`



STEP 4

필터링

조건으로 정상·이상 구
분

`v[v > 기준]`



STEP 5

통계 요약

평균·표준편차로 데이
터 성격 압축

`.mean() · .std()`

NumPy 기초 단단해짐 → 다음은 Pandas — 표 형태 실제 데이터를 더 편리하게 다루기